

Тема: Асинхронна обробка даних у TypeScript

Мета: налаштування середовища розробки та набуття практичних навичок асинхронного програмування в TypeScript, зокрема: • встановлення Node.js та налаштування TypeScript-проєкту; • використання Promise.all та Promise.race для керування паралельними операціями; • реалізація механізмів повторних спроб (retry) при помилках; • послідовна обробка даних партіями (batch processing); • коректна обробка помилок через try/catch в асинхронному коді.

Хід роботи:

3.1. Встановлення Node.js. Встановити Node.js на робочий комп'ютер.

Уже встановлено

3.2. Налаштування проєкту з TypeScript

Створити новий проєкт та налаштувати TypeScript.

package.json

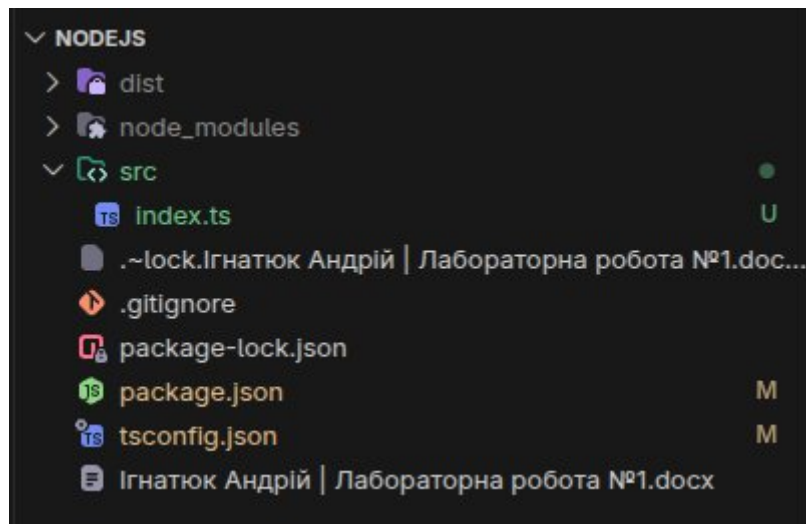
```
{
  "name": "nodejs",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "build": "tsc",
    "test": "echo \\\"Error: no test specified\\\" && exit 1"
  },
  "repository": {
    "type": "git",
    "url": "git+https://github.com/IhnatiukAndrii/nodejs-labs.git"
  },
  "keywords": [],
```

					ДУ «Житомирська політехніка».26.121.8.000 – Лр1			
Змн.	Арк.	№ докум.	Підпис	Дата				
Розроб.		Ігнатюк А.М.			Звіт з лабораторної роботи		Літ.	Арк.
Перевір.		Фуріхата Д.В.						Аркушів
Керівник							1	11
Н. контр.							ФІКТ Гр. ІПЗ-23-1	
Зав. каф.								

```

"author": "",
"license": "ISC",
"type": "commonjs",
"bugs": {
  "url": "https://github.com/IhnatiukAndrii/nodejs-labs/issues"
},
"homepage": "https://github.com/IhnatiukAndrii/nodejs-labs#readme",
"devDependencies": {
  "typescript": "^6.0.3"
}
}

```



```

> nodejs@1.0.0 build
> tsc

```

3.3. Функція delay

```

export async function delay(ms: number): Promise<void> {
  try {
    console.log(`[delay] Start: waiting for ${ms}ms`);

    if (ms < 0) {
      throw new Error("Delay duration cannot be negative");
    }
  }
}

```

		Ігнатюк А.М.			ДУ «Житомирська політехніка».26.121.8.000 – Лр1	Арк.
		Фуріхата Д.В.				2
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    await new Promise<void>((resolve) => {
        setTimeout(resolve, ms);
    });

    console.log(`[delay] Success: completed waiting for ${ms}ms`);
} catch (error) {
    console.error(`[delay] Error: failed during delay.`, error);
    throw error;
}
}

```

3.4. Функція fetchUserProfiles

```

import { delay } from './delay';

export interface UserProfile {
    id: string;
    name: string;
    email: string;
}

export async function fetchUserProfiles(userIds: string[]): Promise<UserProfile[]> {
    try {
        console.log(`[fetchUserProfiles] Start: fetching profiles for userIds [${userIds.join(', ')}]`);

        if (userIds.length === 0) {
            console.log(`[fetchUserProfiles] Success: userIds is empty, returning empty array`);
            return [];
        }

        const fetchPromises = userIds.map(async (id) => {
            try {

```

```

const randomDelay = Math.floor(Math.random() * (150 - 50 + 1)) + 50;

console.log(`[fetchUserProfiles] Simulating fetch for User ${id} (delay:
${randomDelay}ms)`);

await delay(randomDelay);

const profile: UserProfile = {
  id,
  name: `User ${id}`,
  email: `user${id}@example.com`
};

return profile;
} catch (error) {
  console.error(`[fetchUserProfiles] Error fetching profile for User ${id}:`, error);
  throw error;
}
});

const profiles = await Promise.all(fetchPromises);

console.log(`[fetchUserProfiles] Success: loaded ${profiles.length} profiles`);
return profiles;
} catch (error) {
  console.error('[fetchUserProfiles] Error during fetching profiles:', error);
  throw error;
}
}

```

3.5. Функція retryOperation

```
import { delay } from './delay';

export async function retryOperation<T>(
  operation: () => Promise<T>,
  maxRetries: number = 3
): Promise<T> {
  let lastError: unknown;
  for (let attempt = 1; attempt <= maxRetries; attempt++) {
    try {
      console.log(`Спроба ${attempt}...`);
      return await operation();
    } catch (error) {
      lastError = error;
      if (attempt < maxRetries) {
        await delay(100);
      }
    }
  }
  throw lastError;
}
```

3.6. Функція processInBatches

```
export async function processInBatches(
  items: number[],
  batchSize: number,
  processor: (batch: number[]) => Promise<number[]>
): Promise<number[]> {
  try {
    const results: number[] = [];
    const total = Math.ceil(items.length / batchSize);
```

		Ігнатюк А.М.			ДУ «Житомирська політехніка».26.121.8.000 – Лр1	Арк.
		Фуріхата Л.В.				5
Змн.	Арк.	№ докум.	Підпис	Дата		

```

for (let i = 0; i < items.length; i += batchSize) {
    const batch = items.slice(i, i + batchSize);
    const n = Math.floor(i / batchSize) + 1;
    console.log(`Обробка партії ${n}/${total}...`);
    const processedBatch = await processor(batch);
    results.push(...processedBatch);
}

return results;
} catch (error) {
    console.error('[processInBatches] Error processing batches:', error);
    throw error;
}
}

```

3.7. Функція raceWithTimeout

```

export async function raceWithTimeout(
    promise: Promise<unknown>,
    timeoutMs: number
): Promise<unknown> {
    let timer: NodeJS.Timeout | undefined;
    try {
        console.log(`[raceWithTimeout] Start: setting timeout limit of ${timeoutMs}ms`);

        const timeoutPromise = new Promise<never>((_, reject) => {
            timer = setTimeout(() => {
                reject(new Error(`Operation timed out after ${timeoutMs}ms`));
            }, timeoutMs);
        });

        return await Promise.race([promise, timeoutPromise]);
    } catch (error) {
        if (timer) {
            clearTimeout(timer);
        }
        throw error;
    }
}

```

```

const result = await Promise.race([promise, timeoutPromise]);

console.log('[raceWithTimeout] Success: operation completed before timeout');

return result;
} catch (error) {

    console.error('[raceWithTimeout] Error/Timeout:', error);

    throw error;
} finally {

    if (timer) {

        clearTimeout(timer);

    }

}
}

```

Тестування

```

export * from './delay';
export * from './fetchUserProfiles';
export * from './retryOperation';
export * from './processInBatches';
export * from './raceWithTimeout';

import { delay } from './delay';
import { fetchUserProfiles } from './fetchUserProfiles';
import { retryOperation } from './retryOperation';
import { processInBatches } from './processInBatches';
import { raceWithTimeout } from './raceWithTimeout';

if (require.main === module) {

    (async () => {

        console.log('=== Запуск прикладу виконання ===');

        try {

            console.log('\n--- Тестування функції delay ---');

```

		Ігнатюк А.М.			ДУ «Житомирська політехніка».26.121.8.000 – Лр1	Арк.
		Фуріхата Д.В.				7
Змн.	Арк.	№ докум.	Підпис	Дата		

```

await delay(1000);

console.log('\n--- Тестування функції fetchUserProfiles ---');
const profiles = await fetchUserProfiles(['1', '2', '3']);
console.log('Отримані профілі:', profiles);

console.log('\n--- Тестування функції retryOperation ---');
let attempts = 0;
const unreliableOperation = async (): Promise<string> => {
    attempts++;
    if (attempts < 3) {
        throw new Error('Тимчасова помилка');
    }
    return 'Успіх!';
};
const result = await retryOperation(unreliableOperation, 3);
console.log('Результат retryOperation:', result);

console.log('\n--- Тестування функції processInBatches ---');
const numbers = [1, 2, 3, 4, 5, 6, 7, 8];
const processor = async (batch: number[]): Promise<number[]> => {
    await delay(100);
    return batch.map(n => n * 2);
};
const results = await processInBatches(numbers, 3, processor);
console.log('Результат processInBatches:', results);

console.log('\n--- Тестування функції raceWithTimeout ---');
const fast = delay(50).then(() => 'Готово');
const result1 = await raceWithTimeout(fast, 100);
console.log('Результат fast:', result1);

const slow = delay(200).then(() => 'Готово');

```



```

try {
    await raceWithTimeout(slow, 100);
} catch (err) {
    if (err instanceof Error) {
        console.error('Помилка slow:', err.message);
    }
}

console.log('\n=== Запуск прикладу завершено успішно ===');
} catch (error) {
    console.error('Помилка виконання прикладу:', error);
}
})();
}

```

		Ігнатюк А.М.			ДУ «Житомирська політехніка».26.121.8.000 – Лр1	Арк.
		Фвріхата Д.В.				9
Змн.	Арк.	№ докум.	Підпис	Дата		

```

> nodejs@1.0.0 start
> tsc && node dist/index.js

=== Запуск прикладу виконання ===

--- Тестування функції delay ---
[delay] Start: waiting for 1000ms
[delay] Success: completed waiting for 1000ms

--- Тестування функції fetchUserProfiles ---
[fetchUserProfiles] Start: fetching profiles for userIds [1, 2, 3]
[fetchUserProfiles] Simulating fetch for User 1 (delay: 57ms)
[delay] Start: waiting for 57ms
[fetchUserProfiles] Simulating fetch for User 2 (delay: 79ms)
[delay] Start: waiting for 79ms
[fetchUserProfiles] Simulating fetch for User 3 (delay: 68ms)
[delay] Start: waiting for 68ms
[delay] Success: completed waiting for 57ms
[delay] Success: completed waiting for 68ms
[delay] Success: completed waiting for 79ms
[fetchUserProfiles] Success: loaded 3 profiles
Отримані профілі: [
  { id: '1', name: 'User 1', email: 'user1@example.com' },
  { id: '2', name: 'User 2', email: 'user2@example.com' },
  { id: '3', name: 'User 3', email: 'user3@example.com' }
]

--- Тестування функції retryOperation ---
Спроба 1...
[delay] Start: waiting for 100ms
[delay] Success: completed waiting for 100ms
Спроба 2...
[delay] Start: waiting for 100ms
[delay] Success: completed waiting for 100ms
Спроба 3...
Результат retryOperation: Успіх!

--- Тестування функції processInBatches ---
Обробка партії 1/3...
[delay] Start: waiting for 100ms
[delay] Success: completed waiting for 100ms
Обробка партії 2/3...
[delay] Start: waiting for 100ms
[delay] Success: completed waiting for 100ms
Обробка партії 3/3...
[delay] Start: waiting for 100ms
[delay] Success: completed waiting for 100ms
Результат processInBatches: [
  2, 4, 6, 8,
  10, 12, 14, 16
]

--- Тестування функції raceWithTimeout ---
[delay] Start: waiting for 50ms
[raceWithTimeout] Start: setting timeout limit of 100ms
[delay] Success: completed waiting for 50ms
[raceWithTimeout] Success: operation completed before timeout
Результат fast: Готово
[delay] Start: waiting for 200ms
[raceWithTimeout] Start: setting timeout limit of 100ms
[raceWithTimeout] Error/Timeout: Error: Operation timed out after 100ms
    at Timeout._onTimeout (/home/lordenko/ztu/another/nodejs/dist/raceWithTimeout.js:10:24)
    at listOnTimeout (node:internal/timers:588:17)
    at process.processTimers (node:internal/timers:523:7)
Помилка slow: Operation timed out after 100ms

=== Запуск прикладу завершено успішно ===
[delay] Success: completed waiting for 200ms

```

Висновок: в ході виконання роботи налаштував середовища розробки та набуття практичних навичок асинхронного програмування в TypeScript, зокрема: • встановлення Node.js та налаштування TypeScript-проєкту; • використання Promise.all та Promise.race для керування паралельними операціями; • реалізація механізмів повторних спроб (retry) при помилках; • послідовна обробка даних партіями (batch processing); • коректна обробка помилок через try/catch в асинхронному коді.

		Ігнатюк А.М.			ДУ «Житомирська політехніка».26.121.8.000 – Лр1	Арк.
		Фвріхата Д.В.				11
Змн.	Арк.	№ докум.	Підпис	Дата		